Building Chatbot in LangGraph

Dr. Sunil Saumya

Dept of Data Science & Artificial Intelligence
Indian Institute of Information Technology Dharwad

sunilsaumya @iiitdwd.ac.in

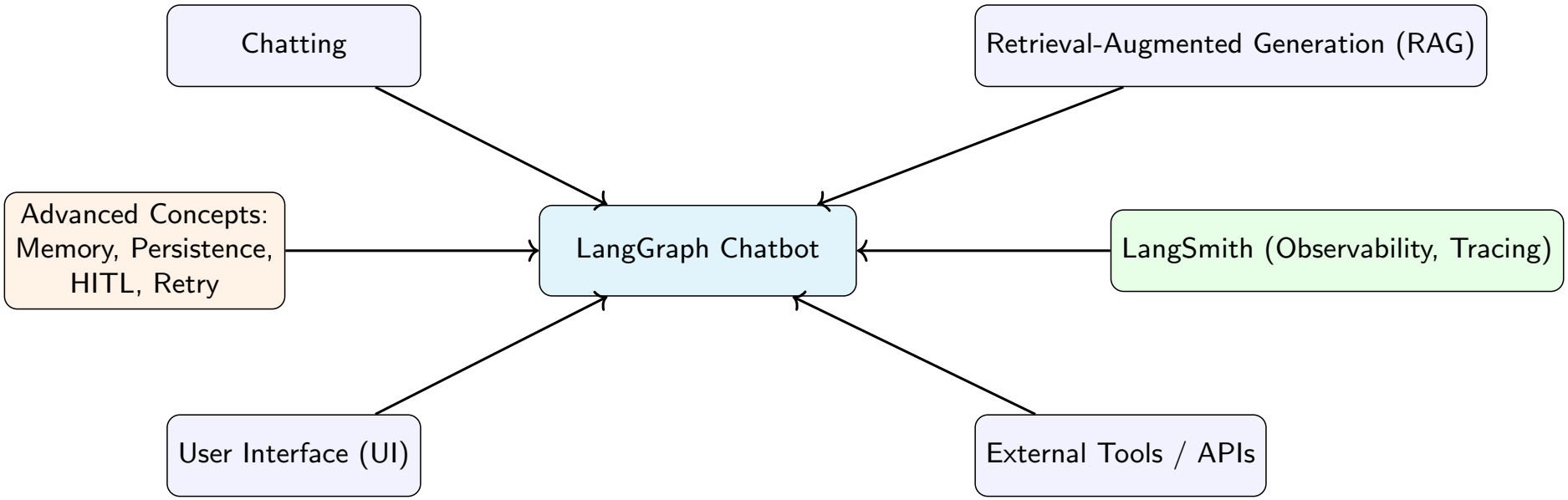


September 6, 2026

Outline

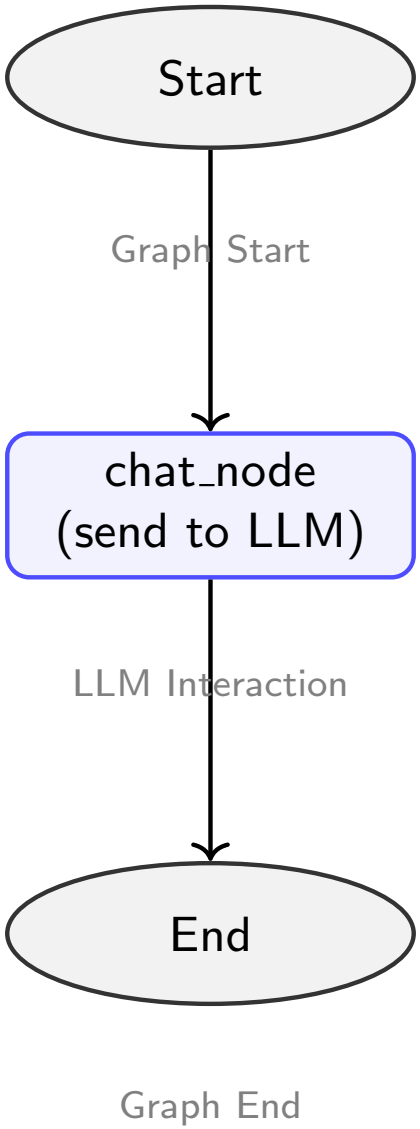
1. Chatbot Architecture
2. Persistence

LangGraph Chatbot Architecture



LangGraph orchestrates LLM workflows with modular, observable, and iterative components

LangGraph Chatbot V1: Workflow



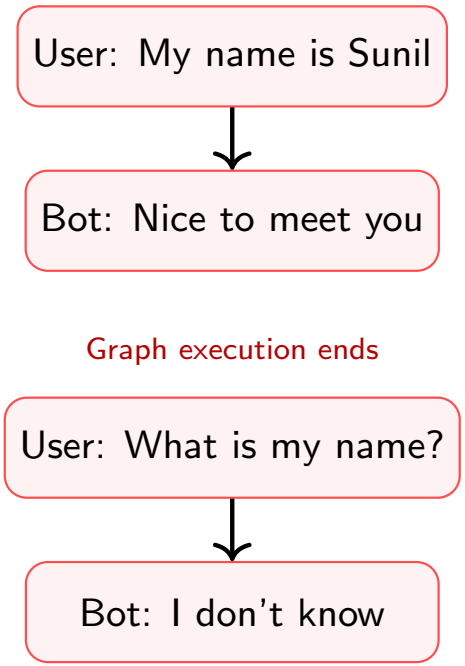
State Evolution

```
# Before
{
  "messages": ["Hi!"]
}

# After chat_node
{
  "messages": [
      "Hi!",
      "Hello! How can I help you?"
  ]
}
```

Demo:- Chatbot V1

Persistence in LangGraph



- Each `invoke()` call processes the current input, generates a response from the LLM, and completes the workflow.
- When we invoke the graph again, the previous state is no longer available and a new state is created.
- Therefore, the new invocation does not have access to the **previous conversation history**.
- The chatbot cannot **continue the conversation from where it left off**.

What Do We Need?

We need a mechanism that can **save the graph state** after an execution and **restore it** when the graph is invoked again.

This mechanism is called Persistence.

With Persistence in LangGraph

Persistence = Saving the Graph State so that it can be restored later

- LangGraph stores the **state** of the graph.
- The stored state can be **retrieved during the next execution**.
- Previous messages, intermediate values, and workflow information can therefore be preserved.
- The graph can effectively **continue from where it left off**.

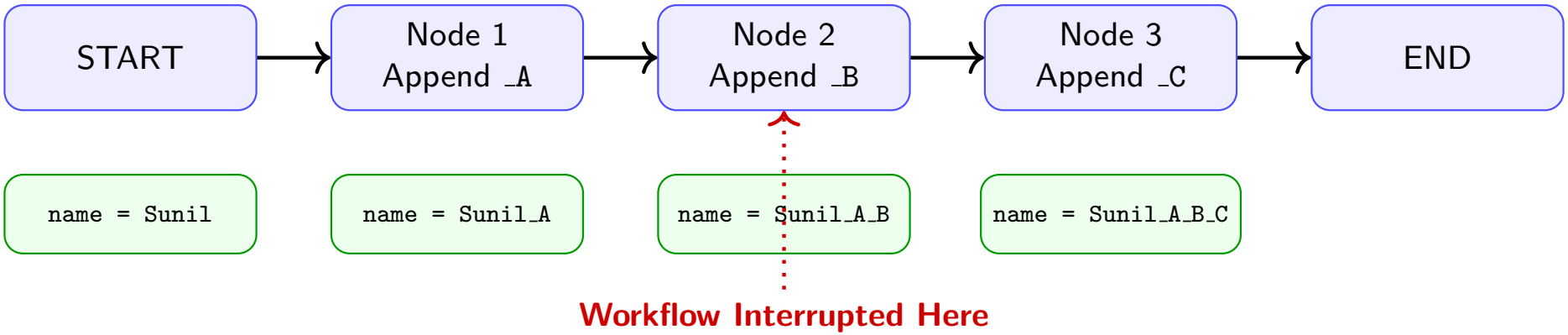
Key Idea

Persistence is not the LLM "remembering" something.

It is the **application storing and restoring the LangGraph state**.

Persistence: What Happens to the State?

Consider a sequential LangGraph workflow with one state variable: `name = "Sunil"`



Without Persistence

State after Node 2: `name = Sunil_A_B`
But when execution stops, this state is lost.

On restarting:

`name = Sunil`

The workflow must start again from the beginning.

With Persistence

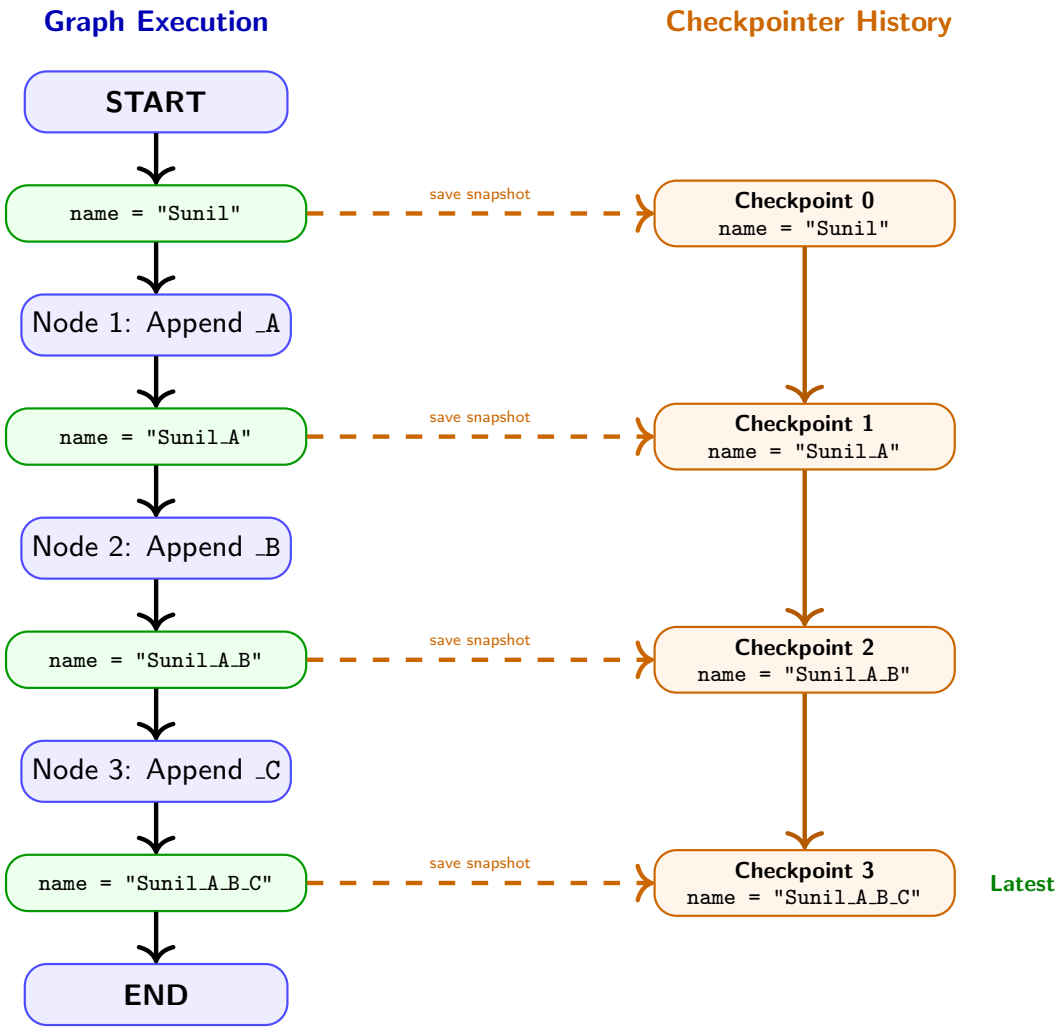
After Node 2, LangGraph saves a checkpoint:

`name = Sunil_A_B`

After interruption, the saved state can be restored.

Checkpointer: Saving the History of Graph State

A checkpointer saves **snapshots of the graph state** as the workflow progresses.



Important Distinction

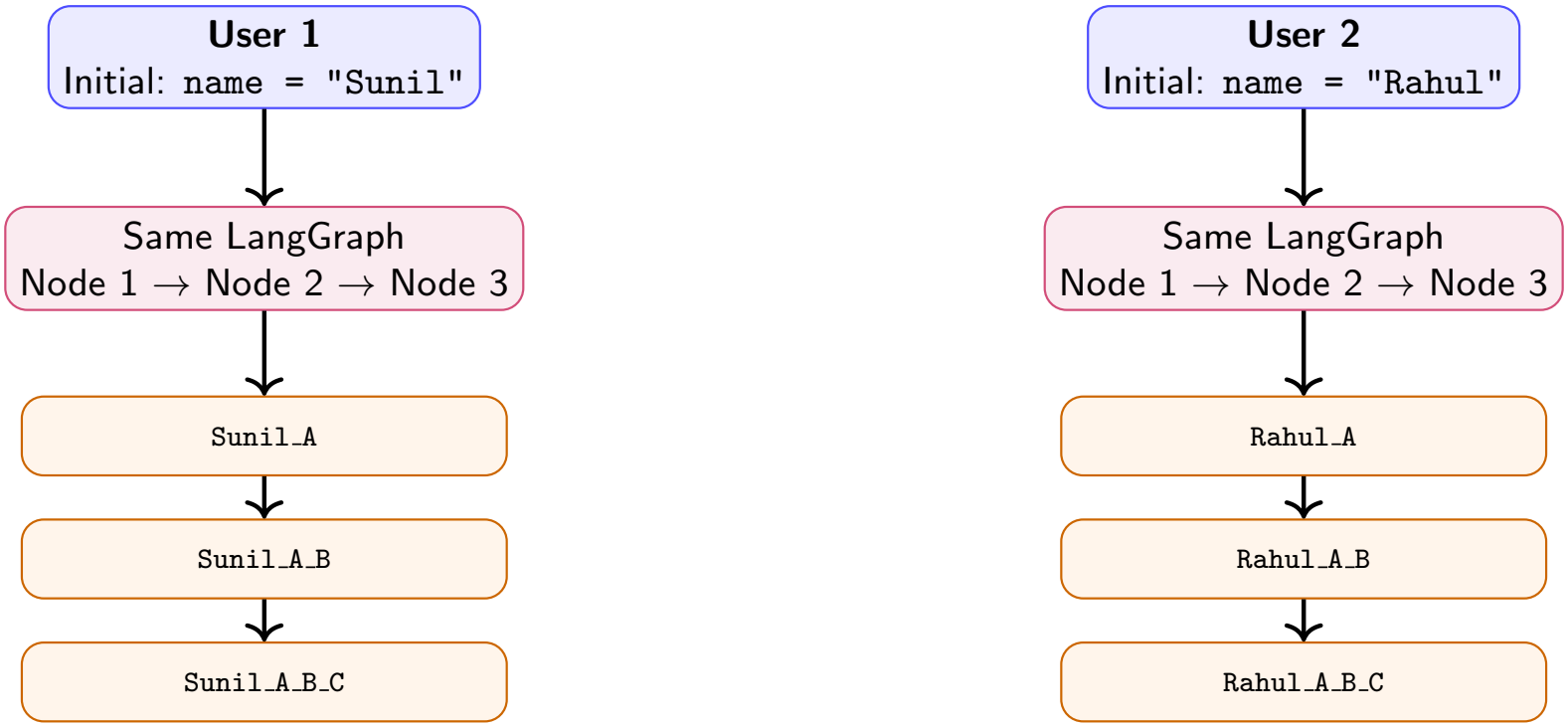
Current State: `name = "Sunil_A_B_C"`

$\neq$

Checkpoint History: sequence of saved state snapshots

Multiple Users: Whose Checkpoint History Is This?

Suppose two users execute the **same LangGraph workflow**. How does the checkpointer keep their state histories separate?

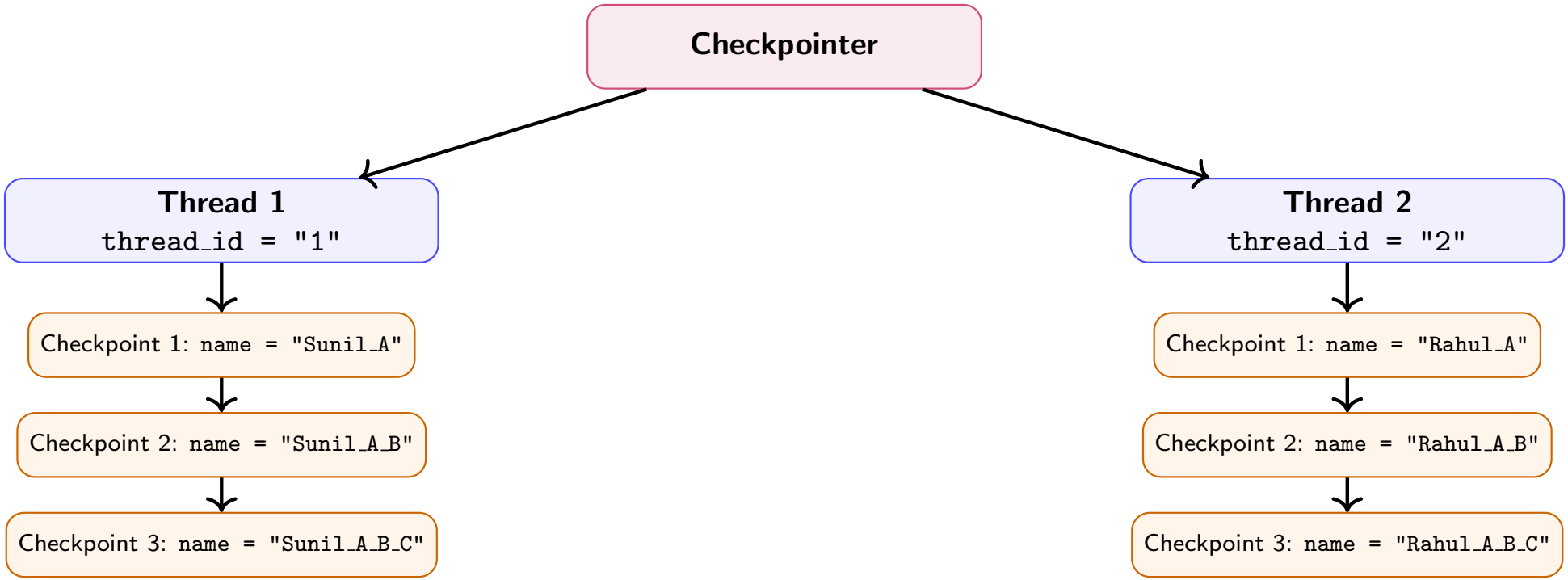


Problem: The checkpointer is storing multiple checkpoint histories.

How does LangGraph know which checkpoints belong to which user/session?

Threads: Organizing Checkpoint History

LangGraph organizes checkpoints into **threads**. Each thread is identified by a unique `thread_id`.



Key Idea

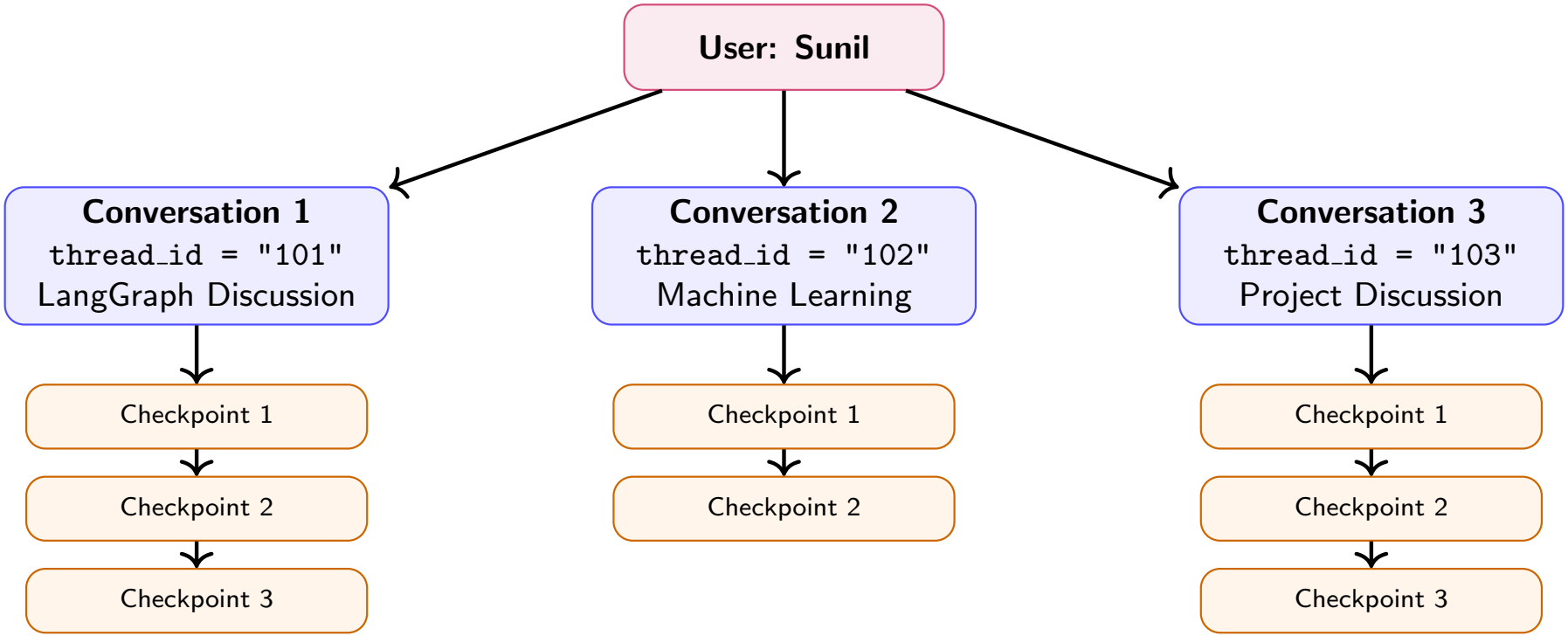
Thread = a sequence of checkpoints associated with the same `thread_id`.

Same `thread_id` $\Rightarrow$ continue with the same persisted state

Different `thread_id` $\Rightarrow$ separate checkpoint history

User vs Thread vs Checkpoint

A user can have **multiple independent conversations**. Each conversation can be maintained as a separate **thread**.



Relationship

One User → Multiple Conversations / Threads → Multiple Checkpoints per Thread

MemorySaver: An In-Memory Checkpointer

`MemorySaver` is a simple LangGraph **checkpointer** that stores checkpoints in memory.

Create a Checkpointer

```
from langgraph.checkpoint.memory import MemorySaver
checkpointer = MemorySaver()
```

Attach it to the Graph

```
graph = builder.compile(checkpointer=checkpointer)
```

What Changed?

Without a checkpointer, graph executions do not have LangGraph-managed persisted checkpoint history.

With `MemorySaver`, LangGraph can save checkpoints for each thread.

Using a Thread with MemorySaver

The `thread_id` tells the checkpointer **which conversation history to retrieve and update**.

Step 1: Select a Thread

```
config = { configurable : { thread_id :  101 }}
```

`thread_id = "101"` identifies one persistent conversation.



Step 2: Turn 1

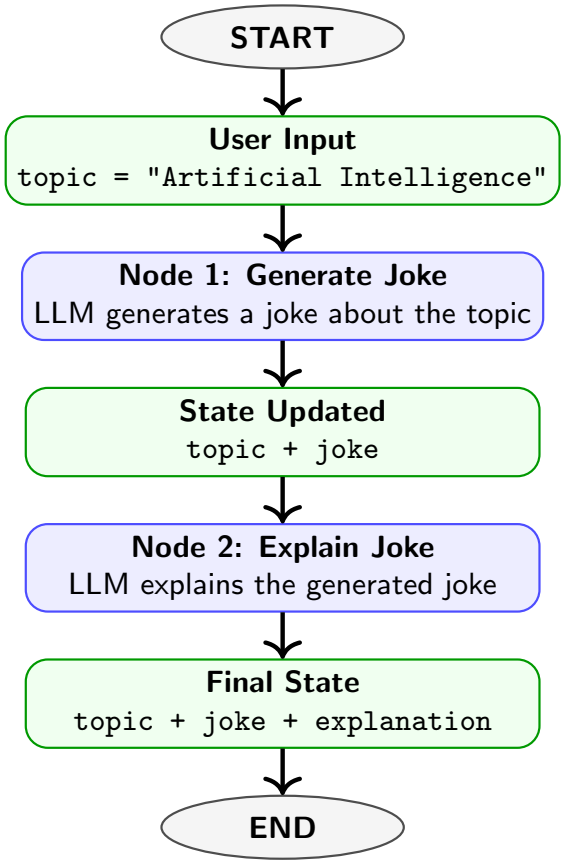
```
graph.invoke(
  { messages : [
    HumanMessage(
      content= My name is Sunil
    )
  ]},
  config=config
)
```

Step 3: Turn 2

```
graph.invoke(
  { messages : [
    HumanMessage(
      content= What is my name?
    )
  ]},
  config=config
)
```

Persistent Coding Demo: Sequential LLM Workflow

We will build a sequential LangGraph workflow where the output of one LLM task becomes the input to the next task.



Persistent Demo

Initial State

```
{ topic: "Artificial Intelligence" }
```

⇓

After Node 1

```
{ topic, joke }
```

The generated `joke` is added to the state.

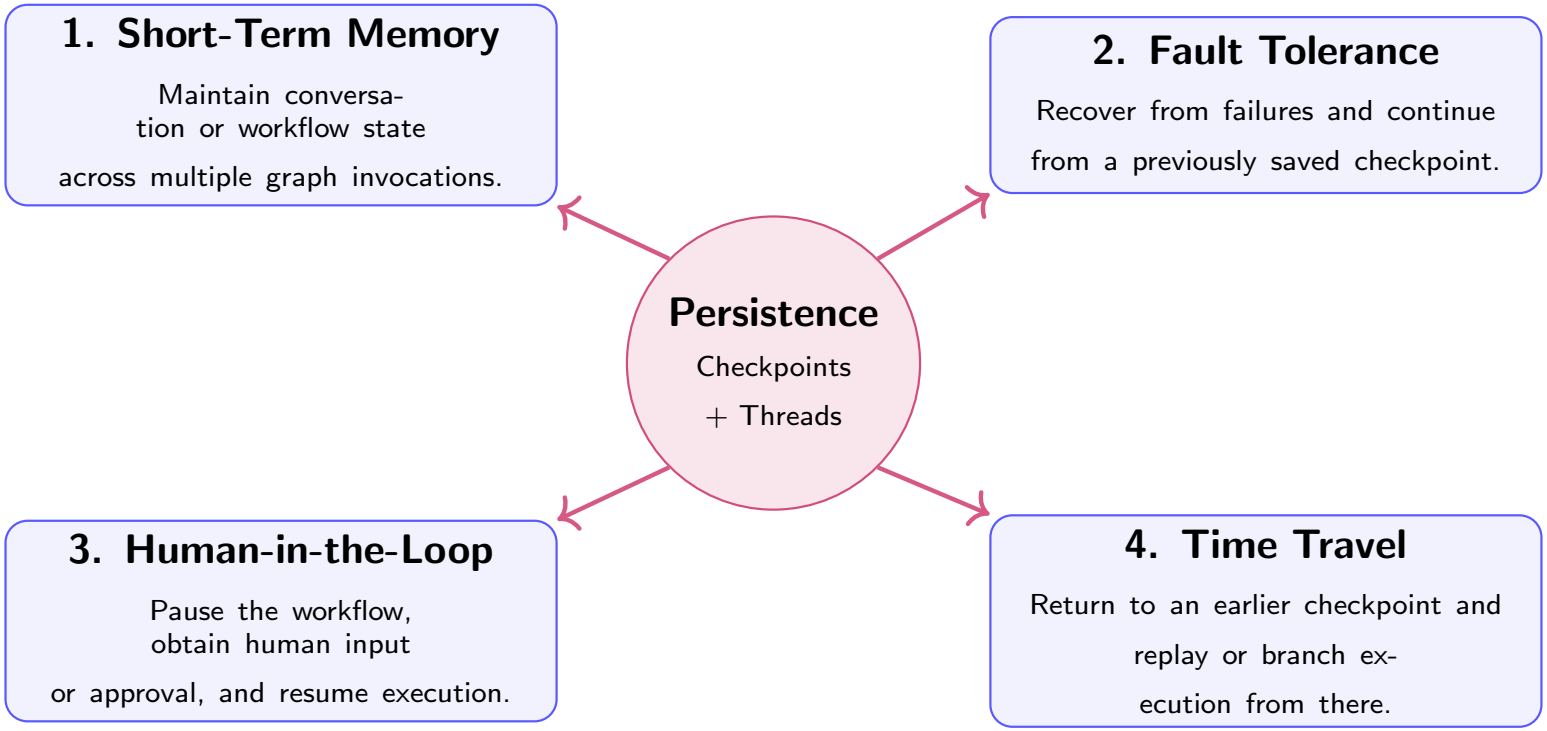
⇓

After Node 2

```
{ topic, joke, explanation }
```

Why Do We Need Persistence?

Persistence enables LangGraph applications to move beyond single, isolated graph executions.



Core Idea

Remember • Recover • Pause & Resume • Revisit

Thank You!